

Comparative Analysis of Machine Learning and Deep Learning Models for Sentiment Analysis

Anmol Sharma

Dept. of Computer Science and Engineering
Chitkara University, Punjab, India
anmol1285.be22@chitkara.edu.in

Devansh Shandilya

Dept. of Computer Science and Engineering
Chitkara University, Punjab, India
devansh261.be22@chitkara.edu.in

Arshaan Singh

Dept. of Computer Science and Engineering
Chitkara University, Punjab, India
arshaan168.be22@chitkara.edu.in

Shreni Kaushal

Dept. of Computer Science and Engineering
Chitkara University, Punjab, India
shreni5047.be22@chitkarauniversity.edu.in

Dr. Shikha Tuteja

Dept. of Computer Science and Engineering Chitkara
University, Punjab, India
shikha.1290@chitkara.edu.in

Abstract—Detecting sentiment in text is now one of the busier problem areas inside natural language processing. The work has direct payoff in places like online review aggregation, the tracking of opinions on social media, and the reading of customer feedback. We put four classifiers head to head on the same problem of polarity detection. Three are well-worn machine learning baselines—Naïve Bayes, Logistic Regression, and a Support Vector Machine (SVM)—and the fourth is a Long Short-Term Memory (LSTM) network. The benchmark used throughout is the IMDb movie review collection. Beyond the usual scoring through accuracy, precision, recall, and F1, we also looked at how long each model took to train and how much it leaned on the hardware. The numbers tell a fairly clear story: the LSTM lands at roughly ninety percent accuracy and tops every other model, while inside the classical group the SVM is the one that holds up best. That said, the older methods are not pushed aside. They keep their value when budgets are small or when someone needs to explain why the model decided what it did. What stands out from the data is a practical tension between accuracy and cost, and the message we take from it is that picking a model is less about prestige and more about matching it to the dataset, the hardware, and what the application actually demands.

Index Terms—sentiment analysis, machine learning, deep learning, LSTM, SVM, natural language processing, text classification

I. INTRODUCTION

Online communication keeps producing text at a pace that is hard to fully describe. Reviews on shopping sites, posts on micro-blogging services, comments under videos, threads on forums—all of it adds up. Sentiment analysis has stepped into this stream as a way of pulling something useful out of it. Companies use it to read what people are feeling, to follow shifts in public reaction, and to ground their decisions in what the text is actually saying. The narrow technical

question that sentiment analysis tries to answer is the polarity of a passage: is it positive, negative, or somewhere neutral?

For a long time the standard recipe was a classical machine learning algorithm sitting on top of features that someone had hand-built—bag-of-words counts, or weights produced by Term Frequency–Inverse Document Frequency (TF-IDF). Naïve Bayes, Logistic Regression, and Support Vector Machines have all been tried in this mode and are still routinely used as baselines. The catch is that none of these methods really sees the order of the words, and they struggle to bring in the surrounding context. Both of those things matter for sentiment, since meaning often hinges on how the words are arranged.

Once datasets grew large and GPUs became easier to come by, the focus moved toward neural models that can pick up their own representations from the raw text. Recurrent networks—LSTMs in particular—were a natural fit for this kind of input because the architecture is built to carry information forward through a sequence. The trade is familiar: better accuracy in many cases, but a heavier bill in compute, memory, and time, both during training and at inference.

What follows is an attempt to put a representative classical model and a representative neural one on the same bench and see how each behaves. Three questions guide the comparison. First, how do the models stack up on the standard classification metrics? Second, what does each cost in computational terms? Third, when should a practitioner reach for one paradigm rather than the other? Putting the answers side by side is, we hope, useful for anyone deciding which kind of model to bring into a sentiment-related project.

II. LITERATURE REVIEW

The study of sentiment classification has come a long way over the past two decades. Early systems leaned on hand-curated lexicons and counting tricks, while the more recent literature is dominated by neural architectures. Pang

and Lee [1] are responsible for one of the early surveys that helped define the area; they laid out the basic toolkit and called attention to harder issues like subjectivity detection and aspect-level reasoning. Liu [2] picked up that thread later and produced a fuller account of the methods and where they were being applied, and his framing has shaped a good portion of the work that followed.

On the classical side, Wang and Manning [7] made a useful argument that simple models are not as outclassed as one might assume. Their finding was that Naïve Bayes, when paired with the right n-gram features, can hold its own against more complicated approaches. Support Vector Machines have been popular in this space because they handle the sparse, very wide feature vectors that come out of text reasonably well. Logistic Regression has stayed in the mix for the simpler reason that it is dependable, calibrated, and easy to reason about.

Deep learning changed the shape of the leaderboards. Kim [4] showed that a fairly small convolutional network running over word embeddings could reach respectable sentence-level accuracy without much hyperparameter fiddling. Recurrent models came into their own in this period too. Drawing on the older LSTM work of Hochreiter and Schmidhuber [5], a number of authors began applying LSTMs to sentiment data, and the architecture's tendency to hang on to context across a sentence proved well matched to the task. The survey by Zhang, Wang, and Liu [8] traced the rapid adoption of these methods and noted that, on large enough corpora, neural models had moved past the older feature-engineered systems.

Word representation has been roughly as important as the model architectures themselves. Maas et al. [9] worked on embeddings that were nudged toward sentiment-aware geometry, an idea that would be carried much further in the era of pre-trained language models. The transformer line eventually arrived, and BERT [6] in particular set a new high water mark across many language understanding tasks, largely on the strength of its bidirectional reading of context. Cambria, Schuller, Xia, and Havasi [3] looked further afield, into affective computing and concept-level work that complements pure polarity detection. Zhang and Wallace [10] gave the field a careful empirical look at how much the convolutional choices matter for text and offered guidance on tuning.

Despite this body of work, head-to-head comparisons of classical and neural models, run under matched conditions, remain a useful service to practitioners trying to pick something to actually deploy. The present paper sits in that gap.

III. OBJECTIVES

The aims of this study can be stated briefly:

- To benchmark the accuracy of representative machine learning models on a sentiment classification task.
- To measure how an LSTM-based deep learning model performs on the same task and the same data.

- To set the two paradigms against one another across accuracy, precision, recall, F1, and measures of computational cost.
- To draw out the situations in which each family of models is the more sensible choice given the trade-offs that emerge from the experiments.

IV. METHODOLOGY

A. Research Design

The work is set up as a quantitative experiment. Each model is fed the same dataset and goes through identical preprocessing and the same evaluation procedure. The point of running things this way is to make sure that whatever differences turn up at the end can be put down to the model itself, and not to differences in how the data was cleaned or split.

B. Dataset

The benchmark for the experiments is the IMDb movie review dataset, which has long served as a reference point for binary sentiment classification. It contains fifty thousand reviews, evenly tagged as positive or negative. We split it down the middle into training and test halves, and from the training portion we further set aside a slice for validation, which is used while training the deep model.

TABLE I
DATA SOURCES CONSIDERED DURING STUDY DESIGN

Source	Description	Access
IMDb Reviews	Movie review corpus for sentiment	Public
Academic Papers	Background literature	Public
Open Repositories	Reference implementations	Public

C. Preprocessing and Feature Extraction

Each review goes through a short cleaning pipeline. The text is lowercased, punctuation and non-alphabetic tokens are dropped, common stop-words are filtered out, and the remaining tokens are passed through a lemmatiser so that inflected forms collapse onto a single base form. From there the two model families need different inputs. The classical classifiers are given TF-IDF vectors built over unigrams. The LSTM, on the other hand, receives sequences of integer token IDs that are then projected into dense embeddings learned alongside the rest of the network during training.

D. Models Under Evaluation

Four classifiers are evaluated in this work—three traditional and one recurrent neural model:

- Naïve Bayes: a probabilistic classifier built on the assumption that the features are conditionally independent given the class.
- Logistic Regression: a linear model that turns a weighted sum of features into a class probability through a sigmoid.

- Support Vector Machine (SVM): a margin-based classifier, well known for behaving sensibly on the kind of sparse, high-dimensional vectors that text produces.
- Long Short-Term Memory (LSTM): a recurrent neural network whose gated cell lets it carry information across long stretches of input.

E. Evaluation Metrics

Performance is reported through the usual quantities that fall out of a confusion matrix.

TABLE II
CONFUSION MATRIX LAYOUT

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

The four metrics used here are written as follows:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (1)$$

$$Precision = TP / (TP + FP) \quad (2)$$

$$Recall = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall) \quad (4)$$

F. Computational Considerations

Accuracy is only part of the picture. Each model was also tracked for how long it took to train, how quickly it could produce predictions, and how much memory it used while doing so. The classical algorithms run comfortably on a regular CPU, whereas the LSTM benefits noticeably from GPU acceleration during training.

G. Methodology Pipeline

Figure 1 lays out the pipeline used in the study, beginning at data collection and ending at the final comparison of results.

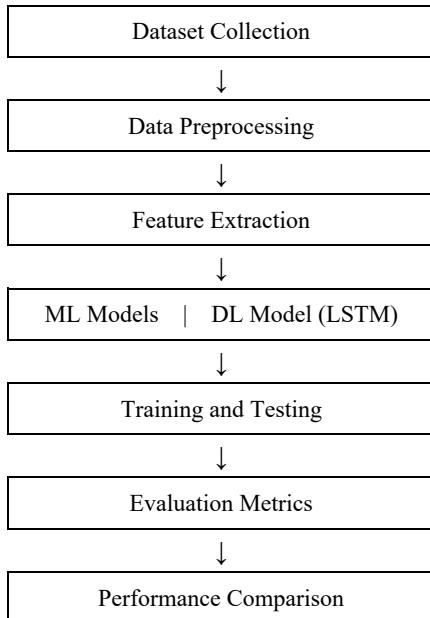


Fig. 1. Methodology workflow of the comparative study.

V. RESULTS AND ANALYSIS

A. Performance Comparison

Table III gathers the classification results obtained on the IMDb test set. Of the three classical models, the SVM comes out ahead. Across all four metrics, however, it is the LSTM that posts the highest numbers.

TABLE III
CLASSIFICATION PERFORMANCE OF EVALUATED MODELS

Model	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	0.82	0.80	0.84	0.82
Logistic Regression	0.85	0.84	0.86	0.85
SVM	0.86	0.85	0.87	0.86
LSTM	0.90	0.89	0.91	0.90

B. Discussion of Machine Learning Models

Within the classical group the SVM did the best, hitting eighty-six percent accuracy. The reason is not particularly mysterious: SVMs are good at drawing decision boundaries through the kind of sparse, high-dimensional spaces that TF-IDF vectors live in, and that fits the geometry of the data well. Logistic Regression sat just below the SVM, which is a useful reminder that a simple, interpretable model is rarely far behind a more elaborate one. Naïve Bayes was the quickest to train of the three but paid the price for its independence assumption, since words in a sentence are obviously not independent of each other.

C. Discussion of the Deep Learning Model

The LSTM recorded ninety percent accuracy, ahead of every classical baseline. The advantage essentially comes from being able to read the order of the words and pick up on context that a bag-of-features representation simply throws away. Looking at the training curves, loss came down steadily across epochs and the validation numbers tracked well, which suggests that the chosen architecture and the regularisation settings were a reasonable match for the size of the training set. The cost, of course, was real: training the LSTM took noticeably more wall-clock time than any of the classical models, and used considerably more memory.

D. Graphical Comparison

Figure 2 turns the accuracy column of Table III into a visual. Two patterns are easy to read off the chart. The LSTM holds a steady lead. And, more subtly, the gap between the SVM and the LSTM is a good deal smaller than the one between Naïve Bayes and the rest of the field.

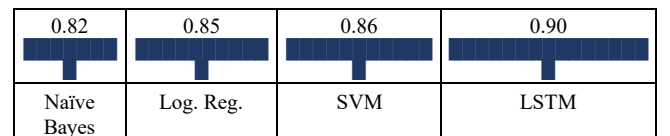


Fig. 2. Accuracy comparison across all evaluated models.

E. Computational Efficiency

Table IV summarises how the four models compare in terms of cost. Here the order flips. Classical methods come out clearly ahead on training speed and memory use, which is what makes them a sensible default in environments where the hardware budget is tight. The LSTM is the heaviest of the four, but the extra investment buys the highest accuracy of the lot.

TABLE IV
COMPUTATIONAL PROFILE OF EVALUATED MODELS

Model	Training Time	Resource Usage
Naïve Bayes	Very Low	Low
Logistic Regression	Low	Low
SVM	Medium	Medium
LSTM	High	High

F. Summary of Observations

A few takeaways emerge from the experiments:

- The LSTM gives the best accuracy and is the only model that genuinely captures context.
- Among the traditional classifiers, the SVM is the one to beat.
- Classical models train far more quickly and ask for far less from the hardware.
- Hand-crafted features still drive the traditional methods, while the deep model learns its own representations from the input.

VI. CONCLUSION

This paper has compared a set of machine learning and deep learning approaches on a sentiment classification benchmark, with one eye on accuracy and another on what each model costs to run. The numbers are consistent. The LSTM beats the classical baselines on every reported metric, and it does so because it can build richer contextual representations directly from the text. Within the classical family, the Support Vector Machine was the strongest, with Logistic Regression just behind and Naïve Bayes trailing.

What runs through all of the results is a fairly old tension between accuracy and resource use. Deep learning gives the better predictions, but the classical methods are by no means obsolete; they remain attractive when speed, simplicity, or interpretability is the binding constraint. The decision about which model to deploy, then, is not really a question of which is better in the abstract. It comes down to the size of the dataset, the hardware that is actually available, and what the application is being asked to do.

The broader lesson is that no single approach is the right one in all settings. Each side has its strengths, each has its weaknesses, and the most useful thing a practitioner can do is match the model to the situation. Future work should look at transformer-based models, and at hybrids that try to combine the better parts of both families.

VII. FUTURE SCOPE

A few directions stand out as worth pursuing. The most obvious is to bring transformer-based language models such as BERT and its descendants into the comparison. They have set the bar across most natural language tasks in recent years, and their bidirectional attention is particularly well placed to pick up the subtler cues that drive sentiment.

Hybrid and ensemble setups are another avenue. Mixing handcrafted features with neural representations, or pooling predictions from several different classifiers, often yields a more robust system without driving up the compute requirements too sharply. Aspect-based sentiment analysis is a related extension that is worth exploring; instead of producing one label for an entire piece of text, the system focuses on specific entities or attributes mentioned within it. That kind of granularity is especially useful for product reviews and for working with customer feedback at scale.

Multilingual sentiment analysis is still an open problem. The bulk of the literature is built around English, and languages with fewer resources have had less attention. Cross-lingual transfer is one of the more promising routes for closing that gap. Sarcasm, irony, and figurative language are similarly stubborn; making real progress there will probably need either richer contextual modelling or some way of plugging in outside knowledge.

Finally, model compression—pruning, quantisation, knowledge distillation—offers a practical path toward putting deep models into latency-sensitive or resource-limited deployments. There is also room to combine sentiment analysis with neighbouring tasks like emotion recognition, topic modelling, and opinion summarisation, which would broaden what these systems can say about a given piece of text.

REFERENCES

- [1] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.
- [2] B. Liu, "Sentiment analysis and opinion mining," *Synthesis Lectures on Human Language Technologies*, vol. 5, no. 1, pp. 1–167, 2012.
- [3] E. Cambria, B. Schuller, Y. Xia, and C. Havasi, "New avenues in opinion mining and sentiment analysis," *IEEE Intelligent Systems*, vol. 28, no. 2, pp. 15–21, 2013.
- [4] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. EMNLP*, 2014, pp. 1746–1751.
- [5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [7] S. Wang and C. D. Manning, "Baselines and bigrams: Simple, good sentiment and topic classification," in *Proc. ACL*, 2012, pp. 90–94.
- [8] L. Zhang, S. Wang, and B. Liu, "Deep learning for sentiment analysis: A survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 4, p. e1253, 2018.
- [9] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. ACL: HLT*, 2011, pp. 142–150.

- [10] Y. Zhang and B. Wallace, "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," arXiv preprint arXiv:1510.03820, 2015.
- [11] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv preprint arXiv:1301.3781, 2013.
- [12] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. EMNLP, 2014, pp. 1532–1543.
- [13] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. EMNLP, 2013, pp. 1631–1642.
- [14] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification," in Proc. EMNLP, 2015, pp. 1422–1432.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.
- [16] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. ECML, 1998, pp. 137–142.
- [17] W. Medhat, A. Hassan, and H. Korashy, "Sentiment analysis algorithms and applications: A survey," Ain Shams Engineering Journal, vol. 5, no. 4, pp. 1093–1113, 2014.
- [18] D. Tang, B. Qin, X. Feng, and T. Liu, "Effective LSTMs for target-dependent sentiment classification," in Proc. COLING, 2016, pp. 3298–3307.
- [19] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in Proc. NAACL-HLT, 2016, pp. 1480–1489.
- [20] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in Proc. ACL, 2018, pp. 328–339.